

ESTRUTURAS DE DADOS E ALGORITMOS

Pilhas, Filas e Tabelas de Dispersão

Aula 10 · encontrar qualquer coisa num só passo

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Qual a diferença entre uma pilha e uma fila?

2

Num array circular, como é que o índice "dá a volta" ao chegar ao fim?

3

Quanto custa pesquisar numa árvore Red-Black com n elementos?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

O telemóvel tem 1 000 000 de contactos. Quantos passos para encontrar o número da Ana?

Lista desordenada

1 000 000

procurar um a um

Árvore Red-Black

≈ 20

descer a árvore (Aulas 8–9)

Tabela de dispersão

≈ 1

calcular onde está e ir lá

Como é possível não depender do número de contactos? É o tema de hoje.

Da regra de acesso ao acesso direto

1

Revisão

Pilhas, filas e filas duplas.

2

A ideia

Chave $\rightarrow$ função de hash $\rightarrow$ posição.

3

Colisões

Duas chaves, a mesma posição.

4

Desempenho

$O(1)$ em média; fator de carga.

5

Na prática

HashMap em Java e exercícios.

Pergunta-guia: e se, em vez de procurar, pudéssemos calcular onde está cada coisa?

1

PARTE 1

Revisão

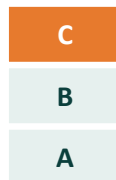
Pilhas, filas e filas duplas.

Três estruturas, três regras de acesso

Pilha

LIFO

`push(x)` · `pop()`



Sai o mais recente.

Fila

FIFO

`enqueue(x)` · `dequeue()`

Sai o mais antigo.

Fila dupla

as duas pontas

`enqueue/dequeue Front e Back`

Pode ser pilha ou fila.

Todas fazem as operações nas pontas em $O(1)$. Mas nenhuma serve para responder depressa a "onde está a chave X?". Para isso, hoje: a tabela de dispersão.

2

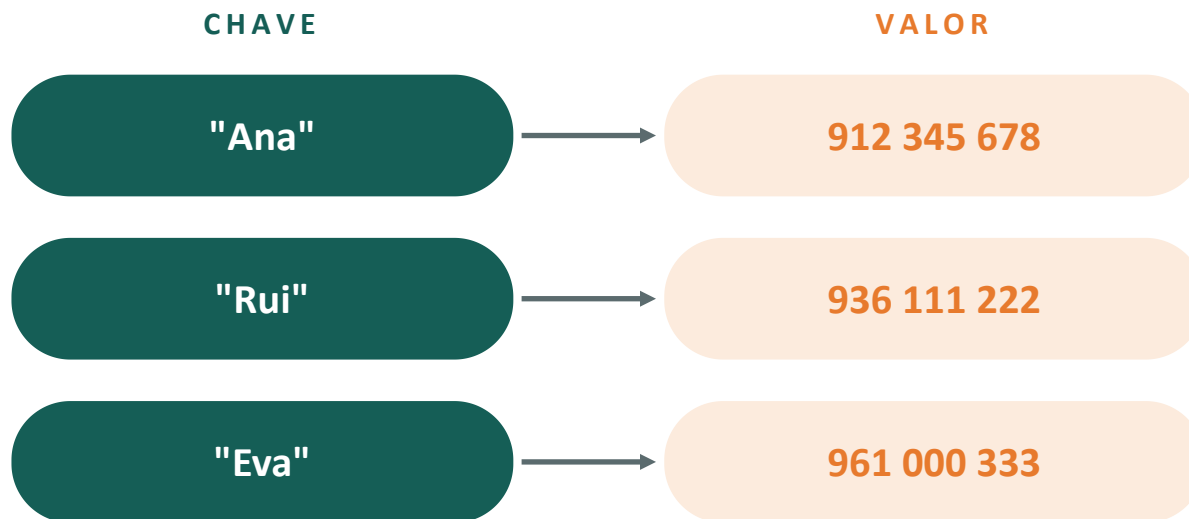
PARTE 2

Tabelas de dispersão

Hash tables: calcular em vez de procurar.

Um dicionário de pares chave → valor

Tabela de dispersão (hash table): estrutura que guarda pares **chave → valor** e encontra o valor de uma chave **muito depressa**.



Como um dicionário

Chave = a palavra que procuramos

Valor = a definição

Ninguém lê o dicionário da primeira página até encontrar a palavra.

A chave diz-nos onde guardar



Guardar: calcular a posição e pôr lá o par.

Procurar: calcular a mesma posição e ir diretamente lá.

O mesmo truque do array circular: o operador % mantém o índice dentro do array.

Três peças

1

Chaves e valores

Cada chave é mapeada para um valor. As chaves são únicas: inserir uma chave que já existe atualiza o valor.

2

Função de hash

Converte a chave num índice. Tem de ser determinística (a mesma chave dá sempre o mesmo índice) e espalhar bem as chaves.

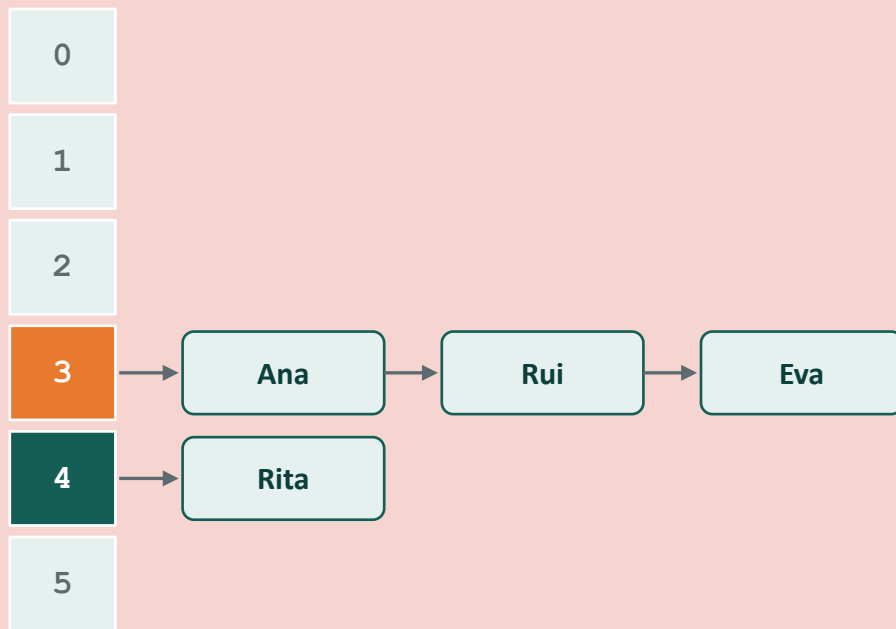
3

Buckets

As posições do array. Cada bucket corresponde a um índice e guarda os pares que lá caem.

Uma boa função espalha; uma má amontoa

Má: $h(\text{chave}) = \text{chave.length}()$



Ana, Rui e Eva têm 3 letras: colidem todos no bucket 3.

Boa função de hash

- ✓ **Determinística**
a mesma chave dá sempre o mesmo índice
- ✓ **Rápida**
calcula-se em tempo constante
- ✓ **Uniforme**
espalha as chaves por todos os buckets
- ✓ **Usa a chave toda**
todos os caracteres contam, não só o tamanho

Inserir, procurar e remover

1

Inserir / atualizar

`put(chave, valor)`

Calcula o bucket e guarda o par. Se a chave já existir, atualiza o valor.

2

Procurar

`get(chave)`

Calcula o bucket e devolve o valor associado à chave (ou nada, se não existir).

3

Remover

`remove(chave)`

Calcula o bucket e retira o par chave-valor.

Passo comum às três: $\text{índice} = h(\text{chave}) \% \text{número de buckets}$. É isto que as torna rápidas.

3

PARTE 3

Colisões

Quando duas chaves querem a mesma gaveta.

Duas chaves, o mesmo índice

Princípio da gaveta

Com mais chaves possíveis do que buckets, alguma gaveta vai ter de receber duas. As colisões não se evitam: gerem-se.

Paradoxo do aniversário

Numa turma de **23** pessoas, a probabilidade de duas fazerem anos no mesmo dia é de **50,7 %**. As colisões aparecem muito cedo!

Então como se resolvem? Duas famílias de soluções:

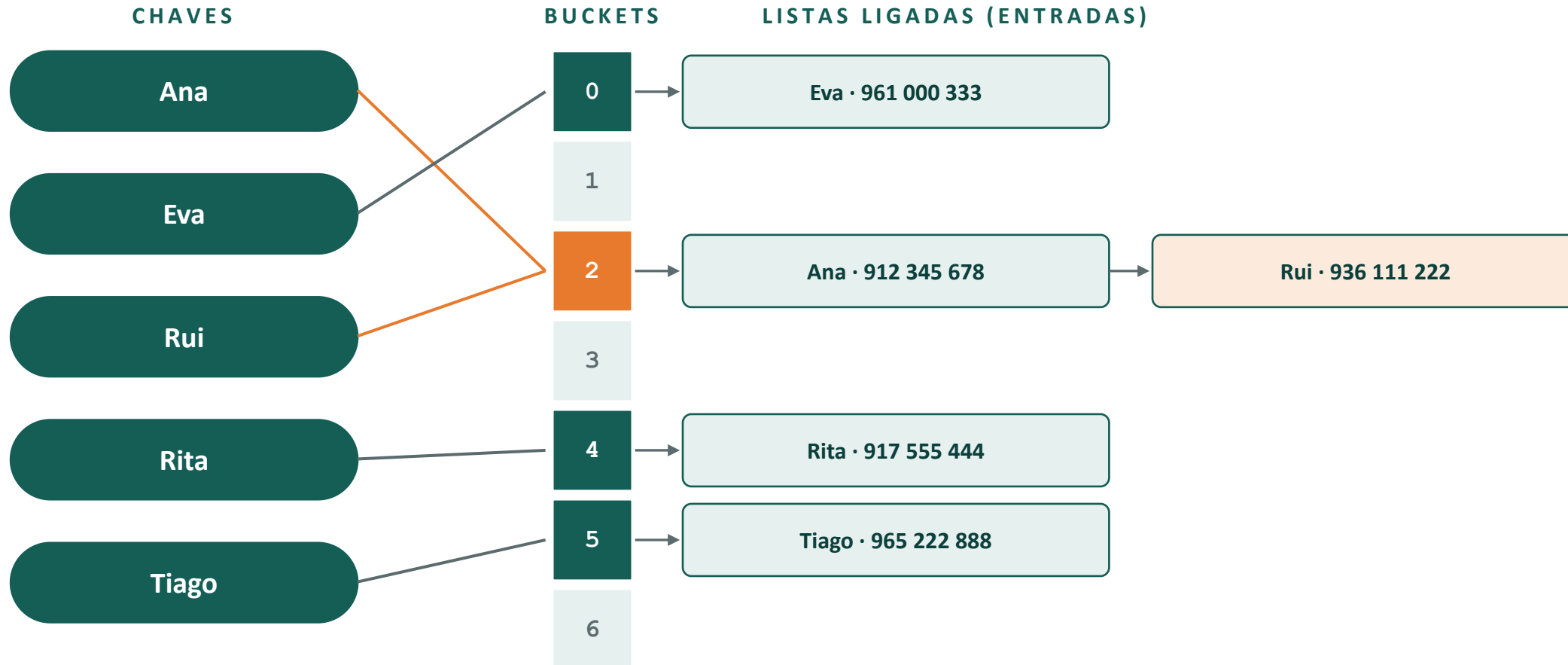
Encadeamento (chaining)

Cada bucket guarda uma lista ligada. É o de hoje e o dos exercícios.

Endereçamento aberto

Se o bucket está ocupado, procura-se outro livre (por exemplo, o seguinte).

Cada bucket tem a sua lista

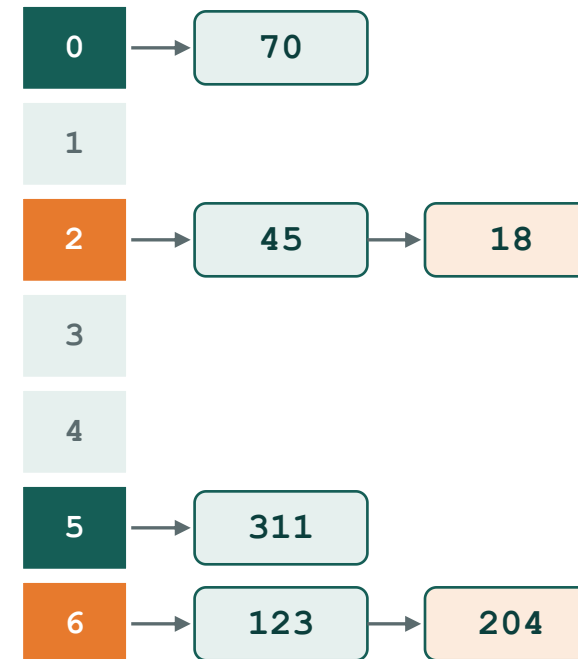


Colisão no bucket 2: Ana e Rui ficam na mesma lista. Os outros buckets têm listas de 1 elemento — ou vazias.

Soma dos dígitos, 7 buckets

Chave	Soma dos dígitos	% 7	
123	$1+2+3 = 6$	6	
45	$4+5 = 9$	2	
18	$1+8 = 9$	2	<i>colisão</i>
311	$3+1+1 = 5$	5	
70	$7+0 = 7$	0	
204	$2+0+4 = 6$	6	<i>colisão</i>

Resultado



Procurar 18: $1 + 8 = 9$, $9 \% 7 = 2 \rightarrow$ bucket 2 $\rightarrow$ compara com 45, depois com 18: encontrado em 2 comparações.

Na mesma tabela (soma dos dígitos, 7 buckets)...

Em que bucket fica a chave 99?

bucket 4

$9 + 9 = 18; 18 \% 7 = 4$

E a chave 1000? Colide com alguém?

bucket 1

$1 + 0 + 0 + 0 = 1; \text{bucket 1, vazio: sem colisão}$

Calcule primeiro no caderno — e só depois veja a resposta.

4

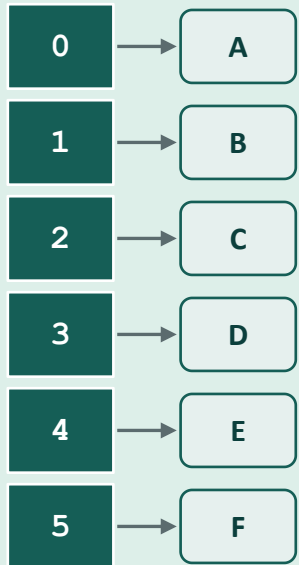
PARTE 4

Desempenho

Rápido em média, lento no azar.

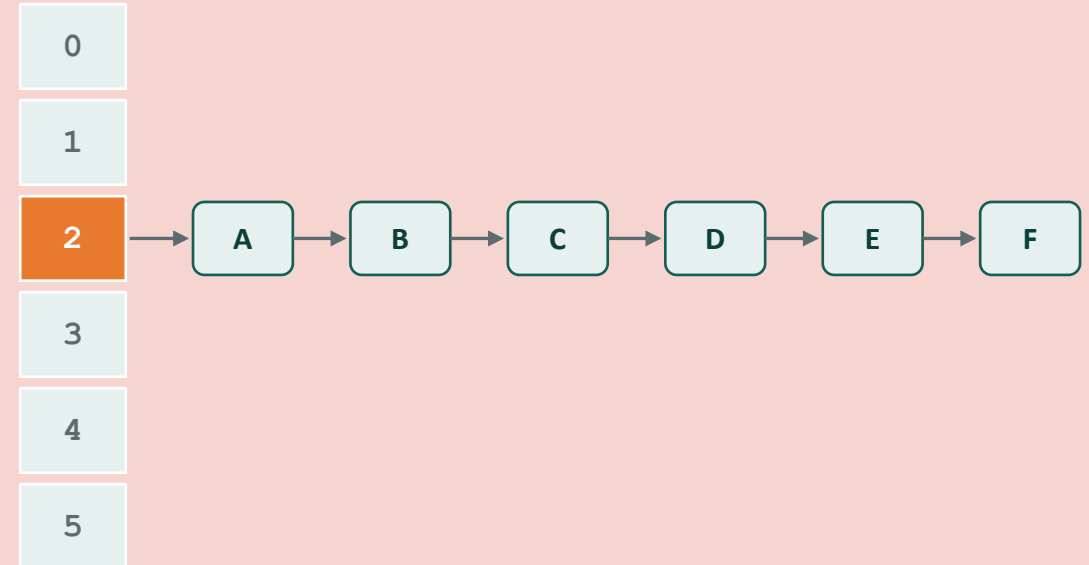
$O(1)$ em média, $O(n)$ no pior caso

Caso médio

 $O(1)$ 

Chaves bem espalhadas: cada lista tem 0 ou 1 elemento.

Pior caso

 $O(n)$ 

Todas as chaves no mesmo bucket: é uma lista ligada disfarçada.

Quão cheia está a tabela?

$$\alpha = n / m$$

n = número de elementos **m** = número de buckets

$n = 6, m = 12$ **$\alpha = 0,5$** *folgada*

$n = 9, m = 12$ **$\alpha = 0,75$** *limite do Java: crescer!*

$n = 24, m = 12$ **$\alpha = 2$** *listas longas: mais lento*

Quando fica cheia: crescer

- 1 Criar um array com o dobro dos buckets
- 2 Recalcular o bucket de cada chave (rehash)
- 3 O fator de carga volta a baixar

Crescer custa $O(n)$, mas acontece raramente: em média, cada inserção continua a custar $O(1)$.

Qual escolher?

	Array ordenado	Árvore Red-Black	Tabela de dispersão
Procurar	$O(\log n)$	$O(\log n)$	$O(1)$ em média
Inserir / remover	$O(n)$	$O(\log n)$	$O(1)$ em média
Pior caso	$O(n)$ ao inserir	$O(\log n)$ garantido	$O(n)$
Mantém a ordem?	Sim	Sim	Não
Em Java	<code>array + binarySearch</code>	<code>TreeMap</code>	<code>HashMap</code>

Regra prática: precisa só de "está lá?" e "qual o valor?" → `HashMap`. Precisa de ordem → `TreeMap`.

Onde se usam tabelas de dispersão

1

Índices de bases de dados

Encontrar um registo pela chave sem percorrer a tabela.

2

Cache (ex.: LRU)

Guardar resultados recentes; junta hash table e fila dupla.

3

Contar ocorrências

Frequência de palavras num texto:
palavra → contagem.

Vantajosas quando...

queremos acesso muito rápido, em média, por chave.

Pouco adequadas quando...

precisamos dos dados ordenados, ou quando há muitas colisões.

HashMap: contar palavras em 4 linhas

```
Map<String, Integer> conta = new HashMap<>();  
for (String p : texto.split(" "))  
    conta.merge(p, 1, Integer::sum);  
  
conta.get("dados");           // quantas vezes?  
conta.getDefault("x", 0);    // 0 se não existir  
conta.containsKey("ED");      // existe?
```

Cada objeto tem hashCode()

`"Ana".hashCode()`

→ 65972

O HashMap usa-o para escolher o bucket.
Chaves iguais têm de ter o mesmo hashCode.

Armadilha (exercício 9): hashCode() pode ser negativo, e `Math.abs(Integer.MIN_VALUE)` continua negativo! Prefira `Math.floorMod(h, m)`, que dá sempre um índice entre 0 e `m - 1`.

5

PARTE 5

Exercícios

Das filas às tabelas de dispersão.

Pilhas e filas: retomar a Aula 7

1

Pilha com array (capacidade 1000): push, pop, size

2

reverse(p, 3): {1,2,3,4,5} → {3,2,1,4,5}

3

Fila com array (capacidade 1000): enqueue, dequeue, size

4

Juntar duas filas ordenadas: [2,4,8,10] + [1,4,9]

5

Variante do 3 com os elementos por ordem crescente

6

Variante do 3 com filas duplas

7

Variantes do 1 e do 3 com LinkedList

Pistas e custos de cada um: diapositivos da Aula 7.

A sua tabela de dispersão

8

Tabela de dispersão de inteiros

- Chaves e valores inteiros
- Um array para os buckets
- Colisões por encadeamento
- Só os métodos para adicionar e obter
- Hash = soma dos dígitos da chave, ajustada ao número de buckets

Teste com o exemplo resolvido: 123, 45, 18, 311, 70, 204 em 7 buckets.

```
int hash(int k) {  
    int soma = 0;  
    k = Math.abs(k);  
    while (k > 0) {  
        soma += k % 10;  
        k /= 10;  
    }  
    return soma % buckets.length;  
}
```

Generalizar passo a passo

9

Chaves e valores String



Refaça o 8 com Strings. Hash: valor absoluto de hashCode() da String, ajustado ao número de buckets.

Cuidado:
`Math.abs(Integer.MIN_VALUE)` é negativo! Use `Math.floorMod`.

10

Remover



Acrescente um método que elimine elementos da tabela.

Basta remover o par da lista do bucket certo.

11

Qualquer tipo



Chaves e valores de qualquer tipo. A função de hash fica ao seu critério.

Pista: genéricos `HashTable<K, V>` + `hashCode()` e `equals()`.

Três ideias a reter

1

Calcular, não procurar

$\text{índice} = h(\text{chave}) \% m$: vai-se diretamente ao bucket.

2

Colisões

São inevitáveis. Com encadeamento, cada bucket guarda uma lista ligada.

3

$O(1)$ em média

Com boa função de hash e fator de carga baixo. Pior caso $O(n)$. Não guarda ordem.

Precisa de rapidez por chave? HashMap. Precisa de ordem? TreeMap.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.^a ed.). Pearson.

2

Areias, M. (2023/24). *Estruturas de Dados — Aula prática 7*. DCC-FCUP. dcc.fc.up.pt/~miguel-areias/teaching/2324/ed/praticas/aula07.html

3

Imagem de referência da tabela de dispersão com encadeamento: dev-to-uploads.s3.amazonaws.com/uploads/articles/rf9k7qhjpwb1w3nb4x46.png

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt